UCLA Samueli School of Engineering

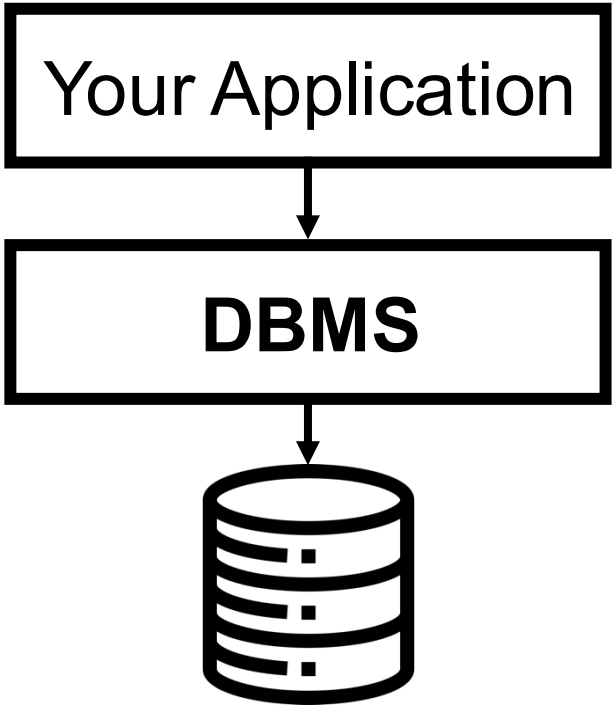
CS 35L Software Construction

Lecture 8 – Data Management

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department

Why do we need a Database Management System (DBMS)?

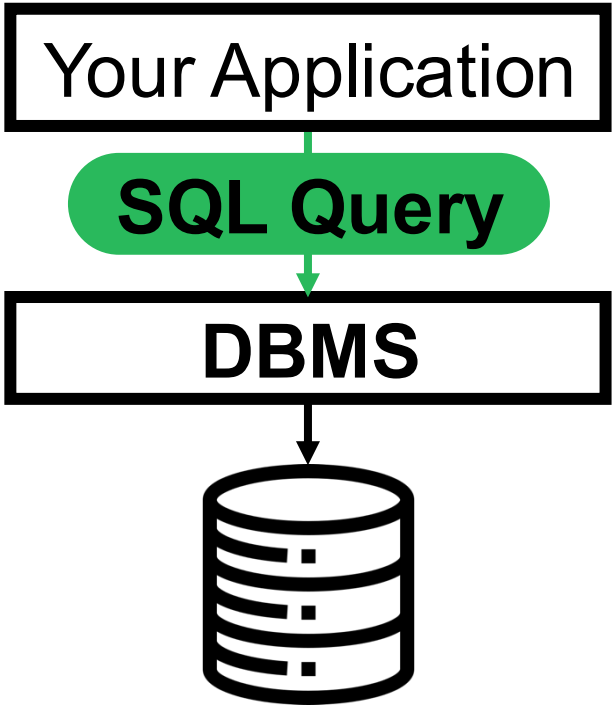
- When storing data **things can go wrong**
  - Power Outage during write operations
  - Multiple simultaneous edits to the same data
- **Redundancy** prevents data loss in the case of disk failures
- **Indexing** optimizes your Data Read speeds
- **Query Optimization** optimizes the speed of your queries



SQL (Structured Query Language) Allows you to read and write data for most DBMS

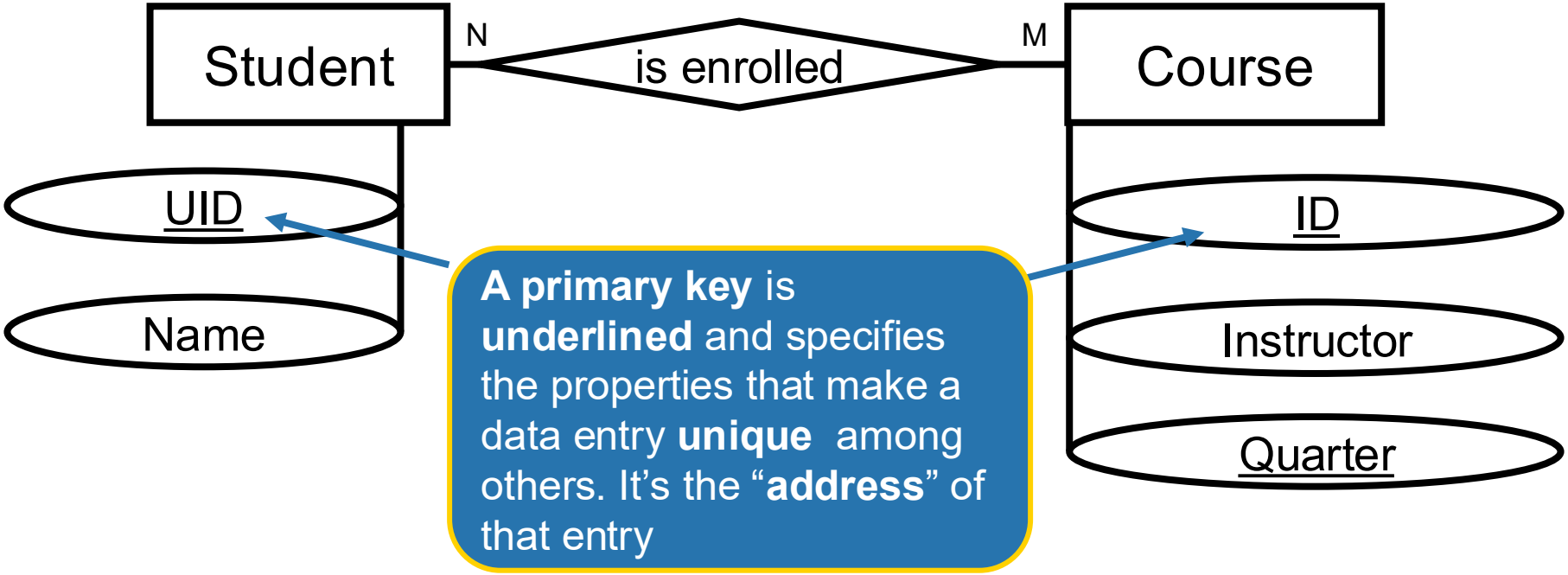
- SQL is **declarative**
  - You specify the operation, the DBMS figures out how to do this efficiently
- The **industry-standard** for almost all DBMS
- Allows you to **swap** out one DBMS for another (mostly)
- Other query languages are **similar to SQL**

In this course, you do **NOT** have to learn SQL syntax. We only show SQL queries for illustrative purposes. But you are welcome to **use SQL in your projects**.



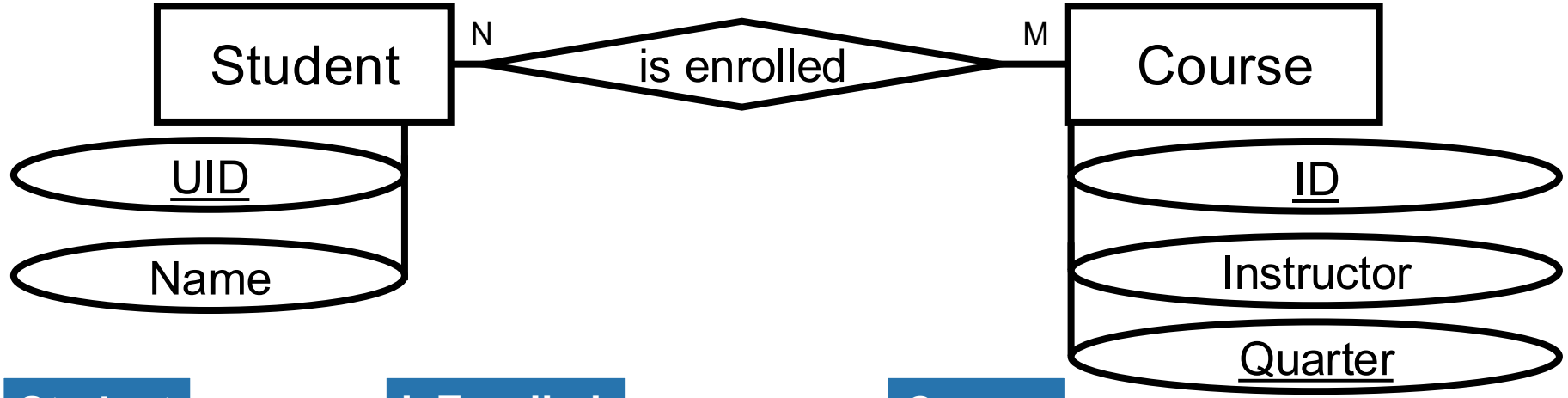
Database Structures are Modeled Using ER Diagrams

Which attributes do we need to store? Talk to your neighbor



Relational Database Management Systems (RDBMS) Store Data in Tables (Relations)

(e.g., Oracle DB, IBM DB2, MySQL, SQLite, PostgreSQL, …)



Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

Relational Database Management Systems (RDBMS) Store Data in Tables (Relations)

(e.g., Oracle DB, IBM DB2, MySQL, SQLite, PostgreSQL, …)

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

SQL Query

```
CREATE TABLE Student (
  uid INTEGER NOT NULL PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);
```

SQL Query

```
CREATE TABLE IsEnrolled (
  uid INTEGER NOT NULL,
  course_id VARCHAR(50) NOT NULL,
  quarter VARCHAR(20) NOT NULL,
  PRIMARY KEY (uid, course_id, quarter),
  FOREIGN KEY (uid)
    REFERENCES Student(uid)
  FOREIGN KEY (course_id, quarter)
    REFERENCES Course(id, quarter)
);
```

SQL Query

```
CREATE TABLE Course (
  id VARCHAR(50) NOT NULL,
  quarter VARCHAR(20) NOT NULL,
  instructor VARCHAR(100),
  PRIMARY KEY (id, quarter)
);
```

A foreign key is a **reference** to the primary key of another data element to link it **consistently**

DBMS Support a Large Variety of Operations

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

e.g.:

- “Give me the names of all students who have taken 35L”
- “Count all students who have taken a course with Remy Wang”
- “For each instructor, count all students who have taken a course with them”

Database Operation: `Join` (R⋈S)

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

- **`Join` combines columns** from one or more tables into a new table, based on one or more related columns between them
  - Looks for matching values in the related columns to combine the rows

Read more here: https://www.geeksforgeeks.org/dbms/joins-in-dbms/

Database Operation: `Join` (R⋈S)

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

Student ⋈ IsEnrolled ⋈ Course

name	uid	quarter	course_id	instructor
Jon Doe	12345	Fall 2025	35L	Tobias Dürschmid
Jon Doe	12345	Fall 2025	110	Remy Wang

Note: Whether the result also contains partial results with null values depends on the type of join used. (**`inner join, outer join, left outer join, right outer join`**)

Read more here: https://www.w3schools.com/sql/sql_join.asp

Database Operation: `Selection` σ

Student ⋈ IsEnrolled ⋈ Course

name	uid	quarter	course_id	instructor
Jon Doe	12345	Fall 2025	35L	Tobias Dürschmid
Jon Doe	12345	Fall 2025	143	Remy Wang

- **`Selection`** uses a Boolean predicate on rows to **get a sub-table:**

$\sigma_{\text{course\_id=35L}}$ (Student ⋈ IsEnrolled ⋈ Course)

name	uid	quarter	course_id	instructor
Jon Doe	12345	Fall 2025	35L	Tobias Dürschmid

Read more here: https://www.w3schools.com/sql/sql_where.asp

Database Operation: `Projection` Π

$\sigma_{\text{course\_id}=35L}$ (Student ⋈ IsEnrolled ⋈ Course)

name	uid	quarter	course_id	instructor
Jon Doe	12345	Fall 2025	35L	Tobias Dürschmid

- `Projection` eliminates columns

$\Pi_{\text{name}}(\sigma_{\text{course\_id}=35L}(\text{Student} \bowtie \text{IsEnrolled} \bowtie \text{Course}))$

name
Jon Doe

Read more here: https://www.w3schools.com/sql/sql_select.asp

Query for: “Give me the Names of all Students who have taken 35L”

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

- **Result:**

$$\Pi_{\text{name}}(\sigma_{\text{course\_id}=35L}(\text{Student} \bowtie \text{IsEnrolled}))$$

name
Jon Doe

Read more about SQL here: https://www.w3schools.com/sql/default.asp

SQL Query

```
SELECT S.name
FROM Student AS S JOIN IsEnrolled AS E
                  ON S.uid = E.uid
WHERE E.course_id = '35L';
```

Projection “Give me the names”

Join to link “all students” with course name

Selection “who have taken 35L”

Query for “Count all students who have taken a course with Remy Wang”

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143
23456	Fall 2024	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
143	Fall 2024	Remy Wang

Step 1: Selection by Instructor Name

$\sigma_{\text{instructor=Remy Wang}}$ (Course)

id	quarter	instructor
143	Fall 2025	Remy Wang
143	Fall 2024	Remy Wang

Query for “Count all students who have taken a course with Remy Wang”

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143
23456	Fall 2024	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
143	Fall 2024	Remy Wang

Step 2: Join with Enrollment to link UID with instructor name

IsEnrolled $\bowtie \sigma_{\text{instructor=Remy Wang}}$ (Course)

uid	quarter	course_id	instructor
12345	Fall 2025	143	Remy Wang
23456	Fall 2024	143	Remy Wang

Query for “Count all students who have taken a course with Remy Wang”

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	143
23456	Fall 2024	143
12345	Fall 2025	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
143	Fall 2024	Remy Wang

Step 3: COUNT DISTINCT on UID

Result

2

SQL Query

```
SELECT COUNT(DISTINCT E.uid) AS Result
FROM IsEnrolled AS E JOIN Course AS C
                     ON E.course_id = C.id
                     AND E.quarter = C.quarter
WHERE C.instructor = 'Remy Wang';
```

Read more here https://www.w3schools.com/sql/sql_count.asp

Query for “For each instructor, count all students who have taken a course with them”

Student

name	uid
Jon Doe	12345
Jane Doe	23456

IsEnrolled

uid	quarter	course_id
12345	Fall 2025	35L
12345	Fall 2025	143
23456	Fall 2024	143

Course

id	quarter	instructor
35L	Fall 2025	Tobias Dürschmid
143	Fall 2025	Remy Wang
32	Fall 2025	David Smallberg

- **Result:**

$\gamma_{\text{instructor},\text{agg}(\text{COUNT})}(\text{IsEnrolled} \bowtie \text{Course})$

Instructor	Students
Tobias Dürschmid	1
Remy Wang	2

Read more here https://www.w3schools.com/sql/sql_groupby.asp

SQL Query

```
SELECT C.instructor, COUNT(DISTINCT E.uid) AS students
FROM IsEnrolled AS E JOIN Course AS C
                     ON E.course_id = C.id
                      AND E.quarter = C.quarter
GROUP BY C.instructor;
```

Group By aggregates all rows who have the same value in the given column (instructor)

A **single, logical unit of work**

Transaction Case Study: Bank Transfer

Accounts

ID	Balance
A	2000

Accounts

ID	Balance
B	1000

Transaction: Transfer 100 USD from Account A to Account B

SQL Query

```
BEGIN TRANSACTION;
      UPDATE Accounts
            SET Balance = Balance - 100
            WHERE ID = 'A';
      UPDATE Accounts
            SET Balance = Balance + 100
            WHERE ID = 'B';
COMMIT;
```

- `BEGIN TRANSACTION;` ← Following steps are in **one Transaction**
- `UPDATE Accounts` ← **Table** to be updated
- `SET Balance = Balance - 100` ← Specification of the **change**
- `WHERE ID = 'A';` ← Specification of the data **rows to be changed**
- `COMMIT;` ← **Saves** changes to the database (ends the transaction)

ACID Properties of Transactions

Atomicity

A transaction is treated as a **single, all-or-nothing unit**. Either all operations succeed, or none of them do; if any part fails, the entire transaction is rolled back.

Case Study: Bank Transfer

Under no conditions does the database end in a state in which account A's balance has been changed without also changing account B's balance.

Read more here: https://www.geeksforgeeks.org/dbms/acid-properties-in-dbms/

ACID Properties of Transactions

Consistency

A transaction must bring the database **from one valid state to another**. This ensures that data adheres to all rules and constraints, preventing invalid data from being written.

Case Study: Bank Transfer

Under no conditions does any changed account violate any of the specified rules (e.g. all balances have to be integers, all balances are non-negative). Otherwise, the transaction is declined.

Read more here: https://www.geeksforgeeks.org/dbms/acid-properties-in-dbms/

ACID Properties of Transactions

Isolation

Concurrent transactions do not interfere with each other. The changes made by one transaction are not visible to other transactions until the first one is complete.

Case Study: Bank Transfer

If simultaneously we calculate the total balance of the entire bank, we either read balances before the transfer or after the transfer, but never in the intermediate state in which only one balance has been changed.

Read more here: https://www.geeksforgeeks.org/dbms/acid-properties-in-dbms/

ACID Properties of Transactions

Durability

Once a transaction is committed, its **changes are permanent** and will **survive any system failures, such as power outages.**

Case Study: Bank Transfer

A power outage happens right after we commit the bank transfer. Fortunately, the system recovers the transaction from the logs to ensure that all balance changes are permanent.

Read more here: https://www.geeksforgeeks.org/dbms/acid-properties-in-dbms/

NoSQL (Not Only SQL) (e.g., MongoDB, Redis, Apache Cassandra) Is an Alternative to Traditional Relational DBMS

- Non-relational database approach that stores and manages data in flexible, non-tabular formats like **documents**, **key-value pairs**, **wide-columns,** or **graphs**
- **RDBMS guarantee ACID**, **NoSQL doesn't** always **guarantee ACID**
  - Many NoSQL databases relax the strict **isolation** levels to **maximize performance**
  - Many NoSQL databases use a weaker **consistency** model (**Eventual Consistency**)
  - Often depends on your configuration of the system

RDBMS

Better for well-structured data with **transactions**

NoSQL

Better for big data that is less well-structured and where **transactions** are **less important**

Case Study for Distributed Database Systems ATMs (Automated Teller Machines)

Consistency

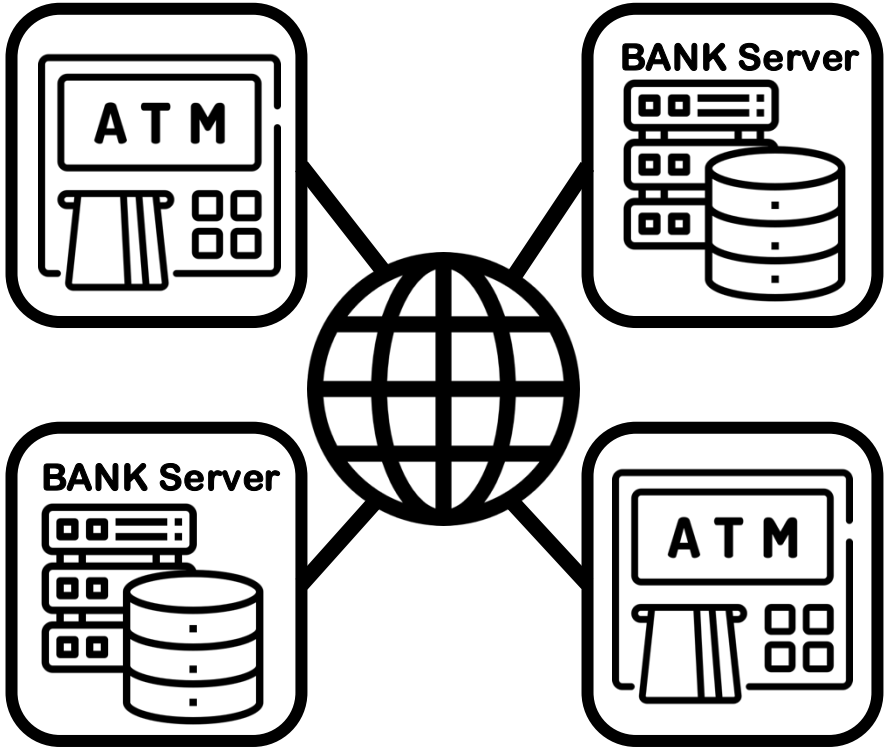
ATMs always show you the correct current balance of your account

Availability

ATMs allow you to withdraw money at any given time

Partition Tolerance

ATMs allow you to withdraw money from different physical locations



Properties of Distributed Database Systems

Consistency

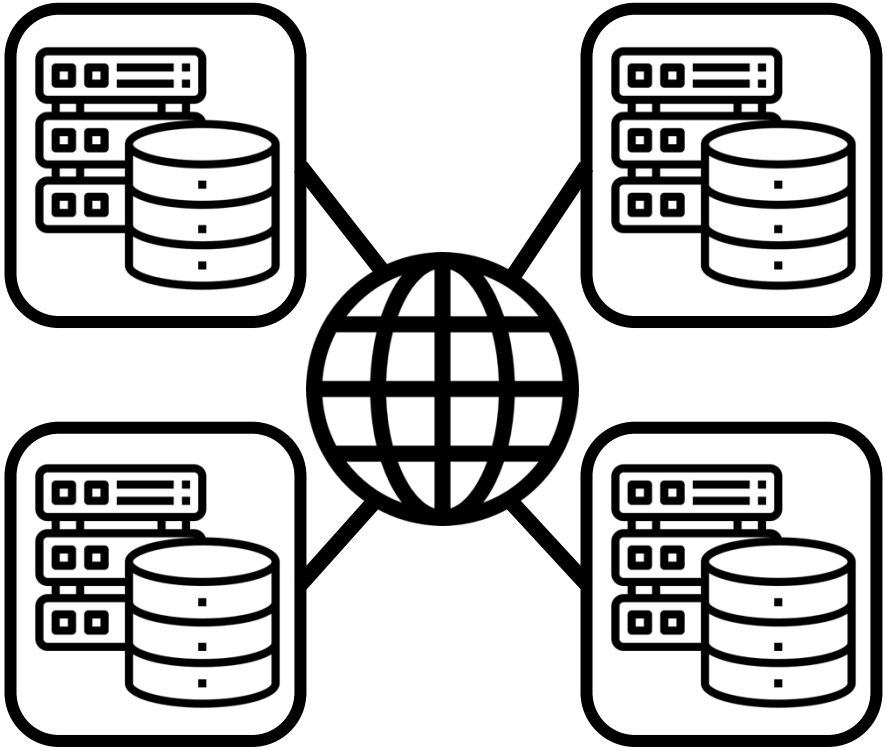
Every read receives the most recent write or an error

Availability

Every read receives a (non-error) response

Partition Tolerance

The system continues to operate despite network partitions



CAP Theorem: You can pick two at most

Consistency

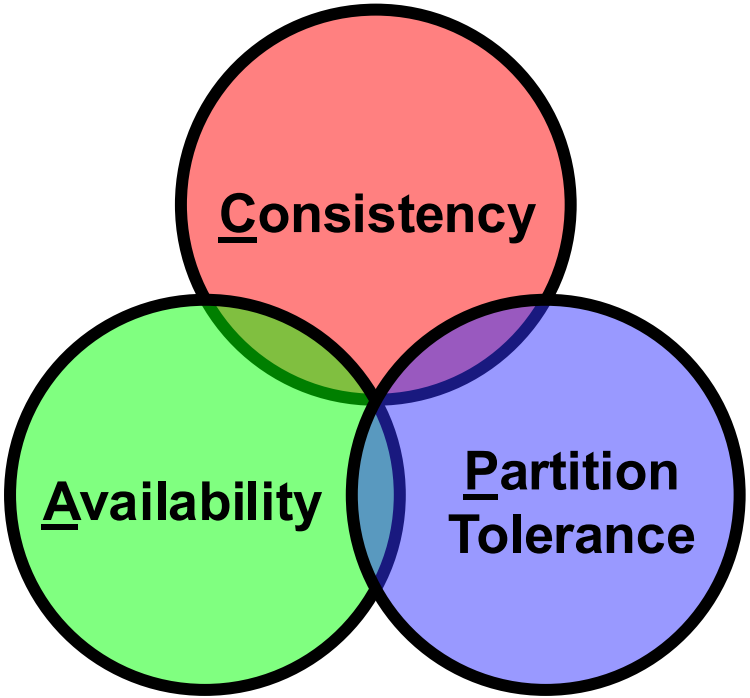
Every read receives the most recent write or an error

Availability

Every read receives a (non-error) response

Partition Tolerance

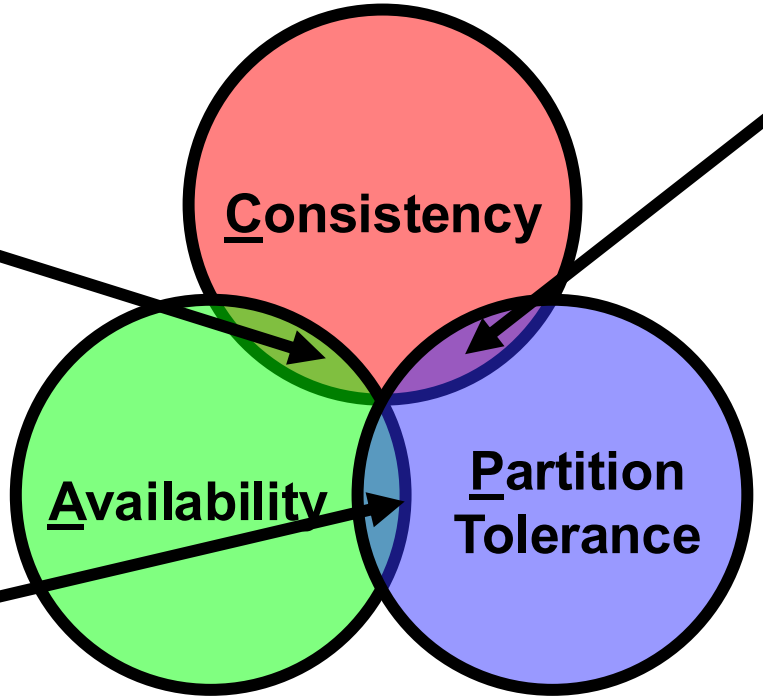
The system continues to operate despite network partitions



Different DBMS make different trade-offs

CA: Traditional Single Node Relational DBMS
(e.g., MySQL, Oracle, PostgreSQL)

AP: Eventually-Consistent DBMS
Most NoSQL databases (e.g., Apache Cassandra, Amazon DynamoDB, CouchDB)



CP: MongoDB
(When configured for **strong consistency** (e.g., using a majority write concern). During a network partition, it will sacrifice availability in the minority partition to ensure the remaining available nodes hold consistent data.

SQL Is Optional in this course! If you like to use it in your project, Use Remy's Tutorial

- https://remy.wang/cs143/notes/sql/sql.html
- I am also working on a SQL tutorial, but it's not ready yet

Please fill out your Exit Tickets on Bruin Learn!

Question 1 — 1 pts

Please summarize **three database operations** you learned today.

Question 2 — 1 pts

Please describe in your own words what the **ACID properties** are and why they are important for data management.

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images from Flaticon.com (Creators: Freepik, phatplus)